

Warm-starting a Motion Planner using Self-Supervised Learning

Timo Matuszewski

*School of Computation, Information and Technology
Technical University of Munich
Munich, Germany*

Gabriel Salinas

*School of Computation, Information and Technology
Technical University of Munich
Munich, Germany*

I. OBJECTIVE

Motion planning in robotics is traditionally formulated as an optimization problem that aims to generate smooth and collision-free trajectories. While classical iterative solvers provide high-precision solutions, their dependence on a good initialization (warm-start) often leads to slow convergence or entrapment in a local minimum, limiting their real-time performance.

To overcome these challenges, we propose a self-supervised learning framework that acts as an intelligent warm-starter for trajectory optimization. Our approach combines the generalization capabilities of Deep Learning (DL) and the reliability of classical solvers. This framework will be applied to a robotic arm for smooth and collision-free motion planning.

II. RELATED WORK

Classical motion planning methods such as CHOMP [1], STOMP [2], and TrajOpt [3] formulate trajectory generation as an optimization problem with costs for collision avoidance and smoothness. While they produce high-quality solutions, their performance strongly depends on initialization and often requires many iterations to converge.

Recent work explores learning-based warm-start strategies to address this limitation. Tenhumberg et al. [4] predict structured initial guesses using deep learning and show improved generalization through suitable environment representations. However, these approaches focus on predicting trajectories directly and do not explicitly incorporate feasibility through optimization-based cost structures.

Other approaches such as Motion Planning Networks (MP-Net) [5] directly predict trajectories, but require supervised datasets generated by classical planners.

Self-supervised inverse kinematics methods [6] demonstrate that feasibility constraints can be learned without labeled data, but focus on single configurations and do not address trajectory initialization.

Our approach bridges these directions by learning structured trajectory representations guided by feasibility constraints and using it to construct structured warm-starts, improving convergence without requiring supervised trajectory data.

III. TECHNICAL OUTLINE

We aim to construct high-quality warm-start trajectories by learning structured trajectory representations that produce near-feasible trajectories.

Inputs and Outputs.

The model takes as input:

- Start and Goal Configurations: $q_{\text{start}}, q_{\text{goal}} \in \mathbb{R}^d$.
- Environment Representation: A discretized representation of the workspace, such as an occupancy grid or signed distance field (SDF).

The network outputs:

- Trajectory Representation: A parameterized trajectory τ , represented either as a sequence of configurations, control points of a spline (e.g. B-spline) or a residual correction of a simple initial trajectory, enabling a compact and learnable representation.

The model learns a mapping:

$$f_{\theta}(q_{\text{start}}, q_{\text{goal}}, env) \rightarrow \tau \quad (1)$$

Self-Supervised Objective. We use a penalty-based formulation to reduce constraint violations along the trajectory:

$$C(\tau) = \sum_{i=1}^N \left(\sum_{b \in B} w_1 C_{\text{coll}}(p(q_i, b), env) + w_2 C_{\text{limits}}(q_i) \right) + w_3 C_{\text{smooth}}(\tau) \quad (2)$$

where:

- B is the set of control points or volumes representing the body parts of the robot.
- $p(q_i, b)$ is the forward kinematics function mapping joint configurations q_i to cartesian positions of a body part b .
- C_{coll} penalizes the proximity of each body part to obstacles.
- C_{smooth} penalizes high-order derivatives (velocity and acceleration).
- C_{limits} enforces the joint limits of the robot.

The network is trained without labeled data by minimizing:

$$\mathcal{L} = C(\tau) \quad (3)$$

This allows the network to learn trajectories that minimize constraint violations without requiring labeled demonstrations.

Trajectory Prediction. At inference time, the model generates a near-feasible trajectory in a single forward pass, which provides the downstream optimizer with a point already close to a local minimum. By doing so, we avoid using traditional global search algorithms.

Optimization. The generated trajectory will be refined using a simple gradient-based optimizer to further improve smoothness and constraint satisfaction.

Experimental Setup. We use a 2D kinematic arm environment with randomly sampled obstacles and differentiable cost functions. We evaluate feasibility improvement, trajectory quality and generalization to unseen environments.

IV. MILESTONES

- **07/05/2026:** Set up a 2D kinematic arm environment with obstacle sampling and differentiable cost functions C_{coll} and C_{limits} ; implement visualization and baseline cost evaluation.
- **14/05/2026:** Define input/output representation $(q_{\text{start}}, q_{\text{goal}}, env) \rightarrow \tau$ and implement environment encoding (e.g. occupancy grid or distance field); design neural network architecture.
- **21/05/2026:** Train a self-supervised model for feasibility correction in single configurations, mapping infeasible poses (e.g., in collision) to nearby feasible configurations $(q, env) \rightarrow \hat{q}$ while minimizing deviation from the original pose; evaluate improvement in constraint satisfaction.
- **28/05/2026:** Extend the model to full trajectory prediction $(q_{\text{start}}, q_{\text{goal}}, env) \rightarrow \tau$ and demonstrate improvement over naive initial trajectories (first quantitative results).
- **04/06/2026:** Implement optimization-based trajectory planning baseline and compare against the learned model (accuracy, robustness).
- **18/06/2026:** Construct warm-start trajectories using learned trajectory prediction; evaluate feasibility and trajectory quality.
- **02/07/2026:** Integrate gradient-based trajectory refinement and analyze convergence speed with and without learned warm-starts.

REFERENCES

- [1] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, “CHOMP: Gradient optimization techniques for efficient motion planning,” in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2009.
- [2] M. Kalakrishnan, S. Chitta, E. Theodorou, P. Pastor, and S. Schaal, “STOMP: Stochastic trajectory optimization for motion planning,” in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2011.
- [3] J. Schulman, J. Ho, C. Lee, I. Awwal, H. Bradlow, and P. Abbeel, “Finding locally optimal, collision-free trajectories with sequential convex optimization,” *International Journal of Robotics Research*, vol. 33, no. 9, pp. 1251–1270, 2014.
- [4] J. Tenhumberg, D. Burschka, and B. Bäuml, “Speeding up optimization-based motion planning through deep learning,” *arXiv preprint arXiv:2311.08345*, 2023.
- [5] A. H. Qureshi, Y. Miao, A. Simeonov, and M. C. Yip, “Motion planning networks,” in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2019.
- [6] Y. Xie *et al.*, “Efficient learning of fast inverse kinematics with collision avoidance,” *arXiv preprint arXiv:2311.05938*, 2023.